

Problema de Cobertura de Conjuntos (SCP)

Uma abordagem memética: Algoritmo Genético com Busca Local

Guilherme Grecco Bononi — RA 134754
Guilherme José Meirelles

2026

1. Introdução e definição do problema

O **Problema de Cobertura de Conjuntos** (*Set Covering Problem* — SCP) é um problema clássico de otimização combinatória, NP-difícil. Dado um conjunto de *linhas* que precisam ser cobertas e um conjunto de *colunas* (cada uma com um custo e cobrindo um subconjunto das linhas), o objetivo é escolher um subconjunto de colunas que cubra **todas** as linhas com o **menor custo total** possível.

Este trabalho resolve o SCP com uma abordagem **memética**: um Algoritmo Genético (AG) de estado estável (*steady-state*) combinado com **busca local** aplicada a cada indivíduo. A implementação está em C++ (C++17) e é dividida em dois arquivos: `arquivo_e_estrutura_de_dados.cpp` (leitura, estruturas, construção gulosa) e `algoritmo_genetico.cpp` (seleção, cruzamento, mutação, busca local e o laço principal).

1.1. Modelagem matemática

Sejam m linhas e n colunas. Para cada coluna j , seja c_j o seu custo e $a_{ij} = 1$ se a coluna j cobre a linha i (0 caso contrário). A variável binária $x_j = 1$ indica que a coluna j foi escolhida. O SCP é formulado como o programa inteiro:

$$\begin{aligned} &\text{minimizar } \sum_{j=1..n} c_j x_j \\ &\text{sujeito a } \sum_{j=1..n} a_{ij} x_j \geq 1, \quad \forall i = 1..m \\ &x_j \in \{0, 1\}, \quad \forall j = 1..n \end{aligned}$$

A restrição garante que cada linha seja coberta por pelo menos uma coluna escolhida (viabilidade); a função objetivo minimiza o custo total.

2. Instâncias e formato dos dados

As nove instâncias ficam na pasta `Tabela/` em formato `.dat`. O cabeçalho traz o número de linhas e colunas; cada linha de dados descreve uma coluna: o 2º campo é o custo e os demais são os índices das linhas cobertas. A integridade foi verificada: em todas as instâncias o número de colunas lidas bate com o cabeçalho e os índices estão sempre no intervalo válido. A tabela abaixo lista as instâncias e o **melhor valor conhecido** (referência do professor), usado como baseline para o cálculo do GAP.

Instância	Linhas	Colunas	Melhor conhecido
Teste_01	50	300	557,44
Teste_02	50	500	537,89
Teste_03	50	700	517,58
Teste_04	100	500	1162,8
Teste_05	100	700	1020,12
Wren_01	200	539	7856
Wren_02	222	5522	13908
Wren_03	219	4990	13780
Wren_04	798	4569	58161

3. Estruturas de dados

Os dados ficam em **duas listas de adjacência complementares**: `tabela_de_dados[c]` guarda, para cada coluna, o custo e a lista de linhas que ela cobre; `lista_de_adjacencia_linhas[l]` guarda, para cada linha, a lista de colunas que a cobrem. Essa dupla indexação é o que torna a busca local eficiente.

A solução em construção é acompanhada por um **estado denso atualizado de forma incremental** (em `adicionar_coluna` / `remover_coluna`, em tempo proporcional ao nº de linhas da coluna, sem recálculo global): `colunas_selecionadas` e `custo_total`; `cobertura_linha[l]` (quantas colunas escolhidas cobrem a linha l); `linhas_descobertas`; e `ganho_coluna[c]` (quantas linhas ainda descobertas a coluna c cobriria). A solução final é guardada de forma **esparsa** (apenas os índices das colunas + o custo); as funções `salvar_solucao` / `carregar_solucao` fazem a ponte denso↔esparso.

4. Construção da solução inicial

4.1. Sete funções gulosas

A cada passo o guloso escolhe a coluna com o melhor (menor) *score*, até cobrir todas as linhas. Foram implementadas sete relações custo×cobertura (em `avaliar_coluna`), onde c = custo e k = número de linhas novas que a coluna cobriria:

Função	Score	Função	Score
1	c	5	$c / (k \cdot \ln k)$
2	c / k	6	c / k^2
3	$c / \log k$	7	$\sqrt{c} / k^2$
4	$c / (k \cdot \log k)$		

4.2. Construção gulosa randomizada (GRASP)

`construcao_gulosa_randomizada(funcao, alpha)` monta a cada passo uma **lista restrita de candidatos** (RCL) com *score* dentro de *menor* + $\alpha \cdot (\text{maior} - \text{menor})$ e sorteia um deles. O parâmetro *alpha* controla o equilíbrio entre qualidade (baixo, perto do guloso puro) e diversidade (alto, mais aleatório).

4.3. Eliminação de redundância

`eliminar_redundancia` remove colunas redundantes (cujas linhas já estão todas cobertas por outras), das mais caras para as mais baratas, reduzindo o custo sem descobrir nenhuma linha.

4.4. Calibração e população inicial

Uma varredura *função* $1..7 \times \alpha \{0,10; 0,20; 0,30\}$ (50 repetições cada, com semente fixa) escolhe automaticamente a melhor combinação `func_global` / `alpha_global` por instância. Como a varredura é determinística, a configuração vencedora é estável e pode ser passada por argumento, pulando a varredura (ganho grande de tempo nas instâncias maiores). `construir_populacao` gera então até 50 soluções **distintas** com essa configuração.

Configuração calibrada por instância (vencedores da varredura):

Instância	func	alpha	Instância	func	alpha
Teste_01..05	3	0,10	Wren_02	1	0,20

Instância	func	alpha	Instância	func	alpha
Wren_01	2	0,30	Wren_03	1	0,10
			Wren_04	3	0,10

5. Algoritmo Genético (steady-state)

O laço evolutivo é **steady-state**: a cada iteração gera-se **um** filho, aplica-se busca local e insere-se na população. É a abordagem que melhor funciona para o SCP (Beasley & Chu).

Seleção (vetor_de_probabilidade + solucoes_sorteadas): roleta proporcional a $1/\text{custo}$ — soluções mais baratas têm mais chance; sorteia dois pais.

Cruzamento (cruzamento): fusão estilo Beasley & Chu — colunas presentes nos dois pais são herdadas direto (consenso); as exclusivas de um pai são herdadas com probabilidade que favorece o pai de menor custo. O filho é reparado gulosamente (cobrindo o que sobrou) e tem a redundância eliminada, garantindo viabilidade.

Mutação (mutacao): com ~10% de chance, adiciona algumas colunas aleatórias e re-elimina a redundância, perturbando a solução.

Substituição (substituir_na_populacao): o filho substitui um indivíduo **aleatório da metade pior** ($\text{custo} \geq \text{mediana}$) e é **rejeitado se for duplicata**. Como o melhor fica abaixo da mediana, ele nunca é substituído (elitismo implícito) e a diversidade é preservada.

Crítérios de parada (combinados): estagnação ($\text{MAX_ESTAGNACAO} = 5000$ iterações sem melhorar o incumbente — principal), máximo de iterações ($\text{MAX_ITER} = 50000$, rede de segurança) e **tempo limite** configurável, que torna justa a comparação entre instâncias.

6. Busca local (memética) — vizinhanças

A busca local é aplicada a cada filho e à população inicial. Combina duas vizinhanças, selecionáveis por um *bitmask* modo_busca_local (1 = A, 2 = B, 3 = A+B):

6.1. Vizinhança A — troca (1,1) guiada por cobertura

Para cada coluna escolhida c , identifica suas **linhas exclusivas** (onde $\text{cobertura_linha} == 1$, ou seja, só c as cobre). Se não houver exclusivas, c é redundante e sai. Caso contrário, procura **uma coluna mais barata** que cubra todas as exclusivas de c e faz a troca. A detecção das exclusivas é $O(\text{linhas de } c)$ graças ao estado incremental. Repete até estabilizar (*first-improvement*).

6.2. Vizinhança B — ruína e reconstrução guiada por desperdício

Elemento de originalidade do trabalho. Define um *score* de desperdício $\text{desperdicio}(c) = \text{peso}(c) \times \text{média}_{\text{linhas de } c}(\text{cobertura_linha} - 1)$, isto é, prioriza colunas **caras** cujas linhas já estão **redundantemente cobertas**. Remove as k colunas de maior desperdício, recobre gulosamente as linhas que ficaram descobertas e elimina redundância. Diferente do LNS aleatório clássico, o operador de destruição é **direcionado** pela informação de sobre-cobertura que a estrutura de dados já mantém. A função *busca_local* intensifica com A, depois alterna $\text{RODADAS_RUINA} = 5$ rodadas de B + A, sempre partindo da melhor solução e guardando a melhor encontrada (esquema de busca local iterada).

7. Validação independente

validar_solucão recalcula cobertura e custo **do zero** (sem usar o estado incremental do *solver*), confirmando que cada solução reportada é viável (cobre todas as linhas) e tem o custo correto. Na campanha completa, **234 execuções foram validadas, 0 inviáveis**. Esse validador foi decisivo durante o desenvolvimento: detectou um bug na mutação (leitura de uma coluna já presente,

corrompendo a contagem de cobertura e gerando soluções inviáveis com custo aparentemente abaixo do ótimo), corrigido com um *guard* mínimo. É ele que garante que os GAPs negativos relatados correspondem a soluções **realmente viáveis e mais baratas** que a referência, e não a artefatos de contagem.

8. Metodologia experimental

O GAP percentual é definido como $GAP = (custo_obtido - melhor_conhecido) / melhor_conhecido \times 100$; valor **negativo** significa que o algoritmo **superou** a referência. A campanha (run_experimentos.ps1) é dividida em três experimentos:

E1 — boxplots de GAP: 10 sementes $\times$ 9 instâncias, configuração calibrada, busca local A+B. Mede qualidade e variância.

E2 — estudo de vizinhanças: modos A, B e A+B nas instâncias rápidas.

E3 — tamanho da população: $pop \in \{20, 50, 100\}$.

9. Resultados

9.1. Execução baseline

Uma execução de referência por instância (Testes a 10 s, Wren a 60 s), todas viáveis pelo validador independente:

Inst.	Custo	M. conhec.	GAP %	Col	func/ α	Iter.	Tempo ms	Valid.
Teste_01	557,44	557,44	0,00	9	3/0,10	13445	1608	50/50
Teste_02	577,25	537,89	+7,32	10	3/0,10	5608	1300	50/50
Teste_03	515,30	517,58	-0,44	9	3/0,10	5000	1454	50/50
Teste_04	1157,08	1162,8	-0,49	19	3/0,10	7738	2224	100/100
Teste_05	1018,67	1020,12	-0,14	19	3/0,10	16666	9476	100/100
Wren_01	7873	7856	+0,22	15	2/0,30	9558	2523	200/200
Wren_02	14979	13908	+7,70	37	1/0,20	6156	25008	222/222
Wren_03	14844	13780	+7,72	35	1/0,10	13695	39356	219/219
Wren_04	58939	58161	+1,34	122	3/0,10	4412	60005	798/798

9.2. E1 — qualidade e GAP (10 sementes, A+B)

Inst.	M. conhec.	Melhor	GAP melhor %	Custo médio	GAP médio %	Desvio	Tempo méd. ms
Teste_01	557,44	557,44	+0,00	568,64	+2,01	17,10	514
Teste_02	537,89	544,60	+1,25	559,48	+4,01	14,73	726
Teste_03	517,58	515,30	-0,44	515,52	-0,40	0,67	1166
Teste_04	1162,8	1157,08	-0,49	1157,08	-0,49	0,00	1246
Teste_05	1020,12	1015,51	-0,45	1022,49	+0,23	6,21	1795
Wren_01	7856	7856,00	+0,00	8113,40	+3,28	85,80	627
Wren_02	13908	14293,00	+2,77	14685,30	+5,59	211,47	8902
Wren_03	13780	14495,00	+5,19	14858,90	+7,83	292,97	9848
Wren_04	58161	57868,00	-0,50	58409,80	+0,43	345,08	43148

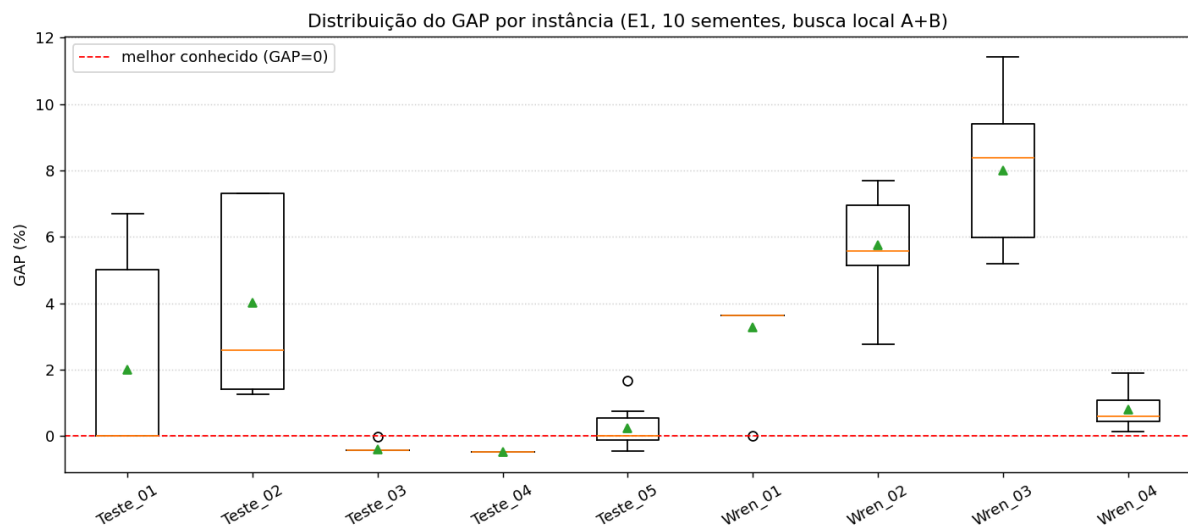


Figura 1 — Distribuição do GAP por instância (E1). A linha vermelha tracejada marca GAP = 0 (melhor conhecido); caixas abaixo dela superam a referência.

9.3. E2 — influência das vizinhanças (GAP médio %)

Instância	A (troca)	B (ruína)	A+B
Teste_01	+3,38	+1,13	+2,23
Teste_02	+4,17	+2,14	+5,50
Teste_03	-0,15	-0,08	-0,37
Teste_04	+0,82	-0,49	-0,49
Teste_05	+2,76	-0,52	+0,36
Wren_01	+3,03	+3,64	+3,03

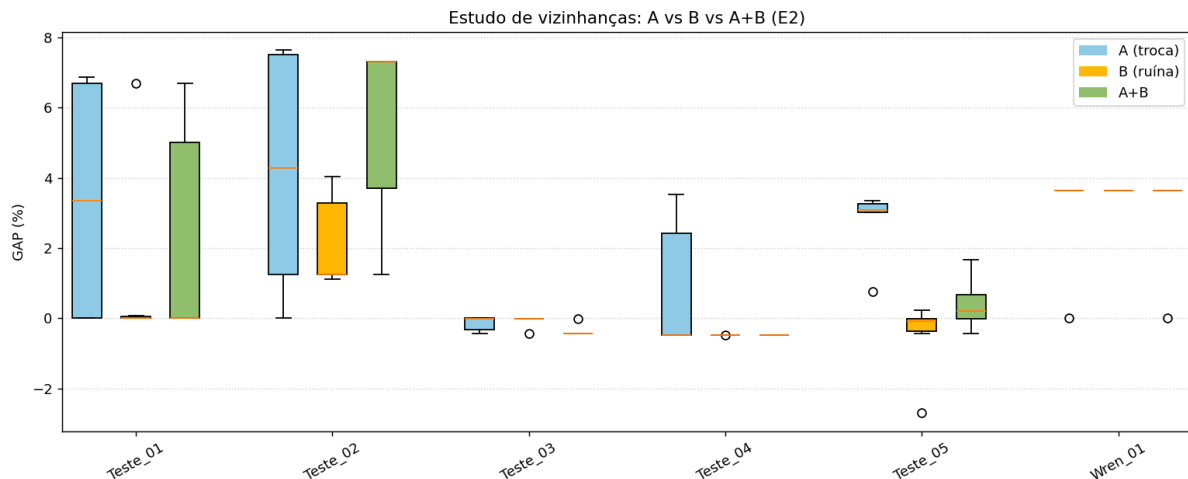


Figura 2 — Estudo de vizinhanças: A vs B vs A+B (E2). As linhas vermelhas dentro de cada caixa são as medianas de cada estratégia.

9.4. E3 — influência do tamanho da população (GAP médio %)

Instância	pop = 20	pop = 50	pop = 100
Teste_02	+4,67	+5,50	+4,39
Wren_01	+3,64	+3,03	+2,43

10. Análise e discussão

Qualidade (E1). As instâncias **Teste_03** e **Teste_04** superam a referência de forma consistente e com desvio praticamente nulo (~ 0), indicando convência estável para um ótimo local muito bom. **Teste_01** e **Wren_01** atingem o ótimo conhecido na melhor semente, e **Teste_05** e **Wren_04** chegam abaixo da referência na melhor semente. As instâncias grandes e densas **Wren_02** e **Wren_03** são as mais difíceis (melhor +2,77% e +5,19%): empacam em ótimo local mesmo com o tempo dado — inclusive parando por estagnação antes do limite. A variância entre sementes justifica o uso de boxplots: em **Teste_01/02** e **Wren_02/03** uma única execução enganaria.

Vizinhanças (E2) — achado central. A vizinhança **B (ruína-reconstrução guiada por desperdício)**, nossa contribuição original, é o principal motor de melhoria: isolada, quase sempre supera a troca A. A troca **A sozinha é a mais fraca**. Curiosamente, **A+B nem sempre supera B isolada** (em **Teste_02** e **Teste_05** chega a piorar), sugerindo que, em algumas instâncias, a troca introduz intensificação que reduz a diversificação útil da ruína.

População (E3). Populações maiores tendem a ajudar nas instâncias grandes: em **Wren_01** o GAP médio cai de 3,64% (pop 20) para 2,43% (pop 100). Em **Teste_02**, instância pequena, o efeito é

misto e dominado por ruído.

11. Conclusões e trabalhos futuros

A abordagem memética (AG steady-state + busca local A+B) resolve bem o SCP nas instâncias de pequeno e médio porte, **superando o melhor conhecido em várias delas** (Teste_03, Teste_04, Teste_05, Wren_04) e atingindo o ótimo em outras (Teste_01, Wren_01). O elemento de originalidade — a ruína-reconstrução guiada por desperdício — mostrou-se o componente mais eficaz da busca local. Todos os resultados foram confirmados por um validador independente (234 execuções, 0 inviáveis).

Trabalhos futuros: (i) melhorar a diversificação em Wren_02/Wren_03 (por exemplo, *restart* ao estagnar ou população maior), que ainda empacam em ótimo local; e (ii) avaliar tornar a vizinhança **B sozinha** o padrão em algumas instâncias, dado o resultado do estudo de vizinhanças.

12. Reprodutibilidade

Compilar e reproduzir a campanha completa:

```
g++ -std=c++17 -O2 algoritmo_genentico.cpp -o scp_run.exe
powershell -ExecutionPolicy Bypass -File Resultados\run_experimentos.ps1
python Resultados\plot_boxplots.py
```

A construção usa semente fixa e os sorteadores usam um gerador mt19937 semeado; fixada a semente, a execução é determinística. A variação entre execuções (para os boxplots) vem da semente passada por argumento.